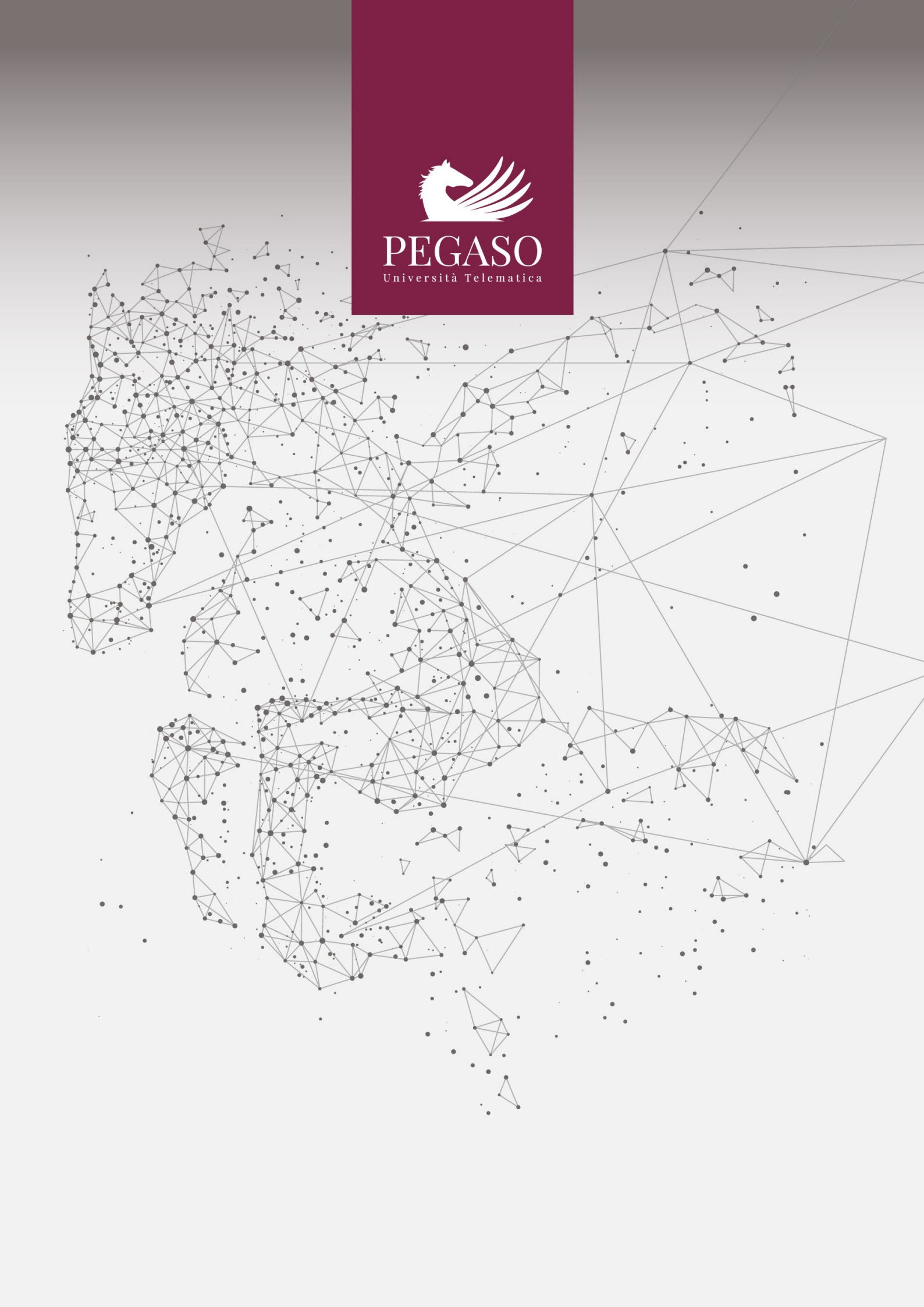

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. SEQUENZA DI ANALISI DI UNA RETE 4
3. INTRODUZIONE A NETWORKX 5
4. GENERAZIONE DI GRAFI CON NETWORKX 7
BIBLIOGRAFIA 17

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presenta la sequenza tipica di analisi di una rete e si introduce Networkx, una libreria Python ad alta produttività per l'analisi di reti complesse. Si illustrano, inoltre, le funzionalità di Networkx per la generazione di grafi sia built-in che personalizzati. Infine, si introducono i metodi per il calcolo dell'ordine e della dimensione di un grafo, per conoscere il vicinato e il grado di un nodo, per assegnare un peso agli archi nei grafi pesati, per generare grafi orientati e per elaborazioni sui grafi come l'unione, il prodotto cartesiano e l'estrazione di sottografi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Sequenza di analisi di una rete

Una tipica **sequenza di analisi di una rete** è costituita dai seguenti passi:

1. si inizia identificando le **entità discrete** e le **relazioni** che le legano. Le *entità* diverranno i **nodi** della rete, mentre le *relazioni* diverranno gli **archi**. Se le *relazioni* hanno una **natura binaria** (presente o assente), esse definiranno direttamente gli archi della rete. Se relazioni sono **continue** o **discrete**, ma non binarie, è possibile trattarle come ***archi pesati*** o convertirle in ***archi non pesati***, ma solo se il valore delle relazioni raggiunge o supera una certa *soglia* (*campionamento*). La **soglia di campionamento** viene scelta sulla base di considerazioni empiriche. Se la *soglia* è troppo alta la rete risulterà troppo sparsa e frammentata in tanti piccoli componenti, mentre se la *soglia* è troppo bassa, invece, la rete perderà ogni struttura comunitaria e diverrà un groviglio;
2. si eseguono vari calcoli sulla rete come la densità, il numero di componenti, la dimensione del GCC (Giant Connected Component), il diametro, le centralità, i coefficienti di clustering;
3. si identificano le **comunità della rete**. Se la rete risulta modulare è possibile assegnare delle *etichette* alle comunità, sostituire le comunità con dei «*supernodi*» e studiare la rete così generata;
4. infine, così come avviene in qualsiasi altro esperimento scientifico, i risultati devono essere **interpretati** e dovrà essere prodotto un **report** con dei grafici.

Il **modulo networkx** fornisce quasi tutto il necessario per ogni attività di studio delle reti con un'importante eccezione riguardante le immagini. Infatti, per ottenere migliori funzionalità di rappresentazione grafica è meglio ricorrere al software **Gephi** che verrà illustrato più avanti.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Introduzione a Networkx

Networkx è una libreria Python ad alta produttività per l'analisi di reti complesse che permette di rappresentare diverse strutture di rete (*orientata, non orientata, multigrafo*). La libreria Networkx è caratterizzata da:

- una flessibilità estrema in quanto i nodi possono essere qualsiasi tipo di oggetto in Python e gli archi possono contenere dati arbitrari;
- un grande numero di **algoritmi** disponibili per i grafi.

Essa, inoltre, è multipiattaforma, facile da utilizzare e fornisce:

1. strumenti per lo **studio della struttura e delle dinamiche delle reti** sociali, biologiche e infrastrutturali;
2. *un'interfaccia di programmazione standard* e un'implementazione grafica adatta a molte applicazioni;
3. un ambiente di rapido sviluppo per progetti collaborativi e multidisciplinari.

A differenza di altri strumenti, Networkx è progettato per **gestire dati con una scala rilevante** per i problemi moderni. La maggior parte degli algoritmi utilizza codice *legacy* estremamente veloce. Inoltre, l'implementazione dei grafi è altamente **flessibile** in quanto i nodi e gli archi possono essere, in pratica, qualsiasi oggetto.

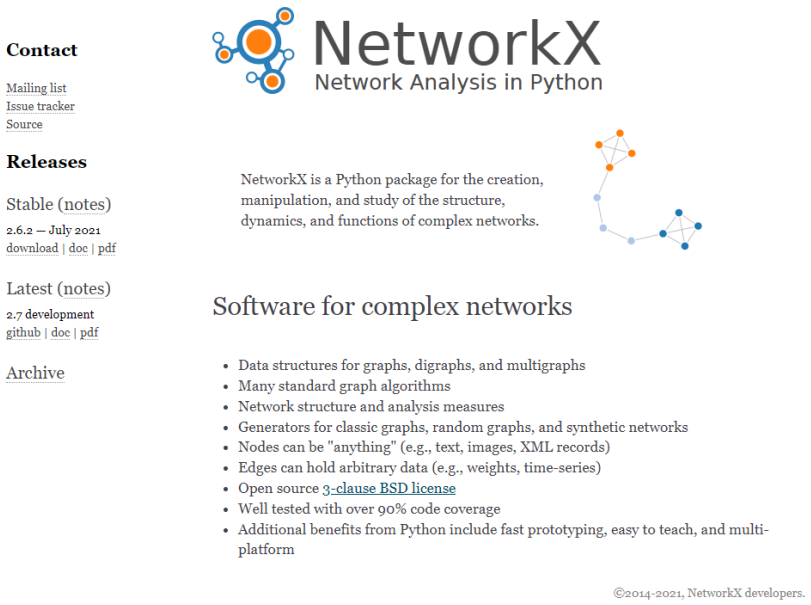
Unica pecca di Networkx è la visualizzazione delle reti, infatti, una migliore visualizzazione dei grafi può essere ottenuta solo con altri strumenti professionali.

Il sito ufficiale di Networkx è disponibile al seguente link: https://networkx.org/

Nella figura seguente si riporta la homepage del sito ufficiale di Networkx.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Per poter utilizzare la libreria Networkx è necessario importarla con la seguente istruzione (ovviamente l'alias può essere modificato):

import networkx as nx

L'istruzione base per la creazione di un **grafo non orientato vuoto** utilizza la funzione **Graph**() di Networkx nel seguente modo:

g = nx.Graph()

Il grafo appena creato può crescere secondo diverse modalità. Networkx, infatti, fornisce diverse funzioni/metodi e utilità per leggere scrivere i grafi in diversi formati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Generazione di grafi con Networkx

Procediamo con l'aggiunta al grafo di semplici nodi numerici. Il metodo per poter raggiungere un nodo è **add_node**() e può ricevere come argomento (nel caso di semplici nodi numerici) un numero singolo. Il codice seguente riporta il numero minimo di istruzioni per importare la libreria, creare un grafo vuoto e aggiungere un nodo numerico.

```
import networkx as nx

g = nx.Graph() # grafo vuoto

g.add_node(1)  # aggiunta di un nodo numerico
```

Il grafo ottenuto a seguito delle istruzioni precedenti risulta essere:



Per aggiungere un insieme di numeri esiste il metodo **add_nodes_from**() a cui può essere passata o una lista di nodi, o un container di nodi come mostrato nel codice seguente:

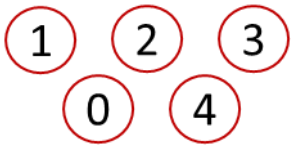
```
g.add_nodes_from([2,3]) # aggiunta di una lista di nodi

c = nx.path_graph(5)    # aggiunta di un container di nodi
g.add_nodes_from(c)
```

Il grafo ottenuto a seguito della prima istruzione del codice precedente risulta essere:



Il grafo ottenuto a seguito della seconda istruzione del codice precedente risulta essere:

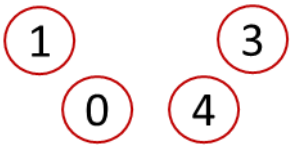


Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per eliminare un nodo dal grafo, invece, si utilizza il metodo **remove_node**() a cui si passa il nodo da eliminare. Nel nostro caso di esempio, i nodi sono numerici e quindi verrà passato un numero al metodo *remove_node*() come mostrato nel codice seguente.

```
g.remove_node(2)    #rimozione di un nodo
```

Il grafo ottenuto a seguito dell'istruzione precedente risulta essere:



Ovviamente all'interno del grafo è possibile aggiungere non solo numeri, ma anche altri **oggetti** quali stringhe, funzioni, file e altri oggetti come riportato nell'esempio seguente in cui vengono utilizzati una stringa, una funzione e un gestore di un file.

```
import networkx as nx
import math

grafo = nx.Graph()
grafo.add_node('stringa')  #una stringa
grafo.add_node(math.cos)   #una funzione
f = open('temp.txt','w')   #un gestore del file
grafo.add_node(f)
print(grafo.nodes())
```

```
['stringa', <built-in function cos>, <_io.TextIOWrapper name='temp.txt' mode='w' encoding='cp1252'>]
```

Per l'inserimento degli archi, una volta creati nodi, esiste il metodo **add_edge**() che prende come argomento i nodi da collegare con l'arco come mostrato nel codice seguente:

```
g.add_edge(1,2)    # aggiunta di un arco
```

Per l'aggiunta di un arco è possibile anche passare una tupla formata da due nodi nel seguente modo (si noti l'uso dell'asterisco per "spacchettare" la tupla):

```
arco = (2,3)
g.add_edge(*arco)    # L'asterisco serve per "spacchettare" la tupla
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Oltre alle modalità appena viste, è possibile anche aggiungere una serie di archi con il metodo **add_edges_from**() a cui si può passare:

- una lista di tuple formate da due nodi ciascuna;
- un container di archi.

Il codice seguente mostra entrambe le possibilità.

```
g.add_edges_from([(1, 2), (1,3)])

g.add_edges_from(c.edges())  # un container di archi
```

Per la rimozione di un arco esiste il metodo **remove_edge**(). I parametri possibili del metodo *remove_edge*() sono gli stessi del metodo *add_edge*(). Si riporta una semplice istruzione che permette di rimuovere l'arco tra i nodi 1 e 2.

```
g.remove_edge(1,2)     #rimozione di un arco
```

Per calcolare l'**ordine del grafo** (che ricordiamo essere il numero di nodi del grafo) è possibile procedere richiamando uno dei seguenti tre metodi:

1. ***number_of_nodes****();*
2. ***order****();*
3. ***len****(g).*

Il codice seguente mostra l'invocazione di tutti e tre i metodi (si noti che len() è una funzione a cui va passato il grafo come argomento e non un metodo del grafo).

```
print(g.number_of_nodes())
print(g.order())
print(len(g))
```

Anche per calcolare la **dimensione del grafo** (che ricordiamo essere numero di archi del grafo) è possibile procedere richiamando uno dei seguenti due metodi:

1. ***number_of_edges****();*
2. ***size****().*

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il codice seguente mostra l'invocazione di tutti e due i metodi.

```
print(g.number_of_edges())
print(g.size())
```

Per ottenere invece l'elenco dei nodi, sotto forma di lista, è possibile ricorrere al metodo **nodes**() del grafo come mostrato nel codice seguente:

```
print(g.nodes())
```

```
[1, 3, 0, 4]
```

Per ottenere l'elenco degli archi si può invocare il metodo **edges**() che visualizzerà a schermo una lista di tuple ciascuna delle quali formata da due nodi corrispondenti alle estremità dell'arco. Si riporta il codice di esempio in cui si richiama il metodo edges().

```
print(g.edges())
```

```
[(1, 0), (3, 2), (3, 4)]
```

Per conoscere i vicini di un nodo è possibile invocare il metodo **neighbors**() a cui va passato il nodo di cui vogliamo conoscere il **vicinato** (i *nodi adiacenti*) come mostrato nel codice seguente:

```
n = g.neighbors(1)
for i in n:
    print(i)
```

```
0
2
3
```

Il **grado di un nodo**, cioè il numero di *archi incidenti* sul nodo di un grafo non orientato è ricavabile attraverso il metodo **degree**() a cui va passato come argomento il nodo di cui vogliamo conoscere il grado come mostrato nel codice seguente:

```
print(g.degree(1))
```

```
3
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Networkx è integrato molto bene con i **dizionari Python** per **memorizzare nodi e archi** (incluse le misure). A solo scopo di ripetizione, si riporta qui un esempio di uso di dizionario Python (dizionario volutamente non omogeneo).

```
diz = {'apple': 1, 'orange': [0.12, 0.02], 42: True}

# Can retrieve the keys and values as Python lists (vector)
print(diz.keys())  # per estrarre le sole chiavi del dizionario

print()

print(diz.items()) # per estrarre le coppie chiave, valore come tupla
```

```
dict_keys(['apple', 'orange', 42])

dict_items([('apple', 1), ('orange', [0.12, 0.02]), (42, True)])
```

Analogamente, ogni nodo di un grafo Networkx si comporta come un dizionario Python dove gli **attributi** dei nodi rappresentano le **chiavi**. Nell'esempio seguente viene associato al nodo 1 l'attributo "time" con valore '10am'.

```
g.add_node(1, time='10am')

print(g.nodes[1]['time'])

d = g.nodes[1]
print(type(d))
print(d) # dizionario Python
```

```
10am
<class 'dict'>
{'time': '10am'}
```

Ovviamente, è possibile attribuire un *attributo speciale* **weight** (***peso***) nei grafi pesati, che dovrà essere numerico al fine di essere utilizzato dagli algoritmi che richiedono proprio gli archi pesati. Nell'esempio seguente, all'arco che collega i nodi 1 e 2 viene assegnato prima peso 4.0 e, successivamente, viene aggiornato a 5.0.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
g.add_edge(1, 2, weight=4.0)
g[1][2]['weight'] = 5.0
print(g[1][2])
```

```
{'weight': 5.0}
```

L'*iterazione* su ciascuno dei nodi e su ciascuno degli archi di un grafo si può ottenere tramite le normali strutture iterative già viste finora e riportate nel codice seguente.

```
print("NODI")
for node in g.nodes():
    print("Nodo:\t",node, "\tgrado nodo:\t", g.degree(node))
print()
print("ARCHI")
for edge in g.edges():
    print("Arco:\t",edge)
```

```
NODI
Nodo:    0      grado nodo:      4
Nodo:    1      grado nodo:      4
Nodo:    2      grado nodo:      4
Nodo:    3      grado nodo:      4
Nodo:    4      grado nodo:      4
Nodo:    a      grado nodo:      0

ARCHI
Arco:    (0, 1)
Arco:    (0, 2)
Arco:    (0, 3)
Arco:    (0, 4)
Arco:    (1, 3)
Arco:    (1, 4)
Arco:    (1, 2)
Arco:    (2, 3)
Arco:    (2, 4)
Arco:    (3, 4)
```

Si propone, di seguito, un ulteriore esempio di *grafo pesato non orientato* con le iterazioni per accedere agli **estremi** di un arco e al suo peso.

```
g = nx.Graph()
g.add_node(1)
g.add_node(2)
g.add_node(3)

g.add_edge(1, 3, weight=2.5)
g.add_edge(1, 2, weight=1.5)
g.add_edge(1, 3, weight=2.5)
g.add_edge(1, 2, weight=1.5)
for n1, n2, attr in g.edges(data=True):
    print("Nodo 1:\t",n1,"\tnodo 2:\t", n2,"\tpeso:\t",attr['weight'])
```

```
Nodo 1:  1      nodo 2:  3      peso:    2.5
Nodo 1:  1      nodo 2:  2      peso:    1.5
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per utilizzare **grafi orientati**, invece, è necessario utilizzare la funzione **DiGraph**() di Networkx.

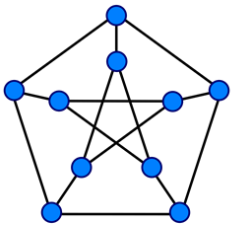
A tal proposito, esistono anche le funzioni **successors**() e **predecessors**() riferite ad un nodo specifico per conoscere il/i nodo/i predecessore/i e successore/i del nodo stesso.

```
dg = nx.DiGraph()
dg.add_weighted_edges_from([(1, 4, 0.5), (3, 1, 0.75)])
print("Grado in uscita del nodo 1:\t",dg.out_degree(1, weight='weight'))
print("Grado del nodo 1:\t",dg.degree(1, weight='weight'))
s = dg.successors(1)
print("Successore/i del nodo 1:\t")
for n in s:
    print(n)
print()
p = dg.predecessors(1)
print("Predecessore/i del nodo 1:\t")
for n in p:
    print(n)
```

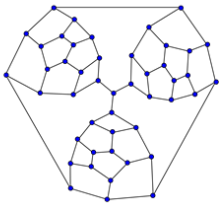
```
Grado in uscita del nodo 1:	0.5
Grado del nodo 1:	1.25
Successore/i del nodo 1:
4

Predecessore/i del nodo 1:
3
```

Come anticipato, Networkx permette di generare dei grafi caratteristici (aventi cioè con una struttura tipica). Si riportano alcuni tra i grafici caratteristici più noti. Per ciascun grafo, vengono dapprima rappresentati i modelli dei grafi e, successivamente, viene riportata la funzione da invocare per ottenere tali grafi insieme al codice per elencare i nodi e gli archi.



Petersen



Tutte

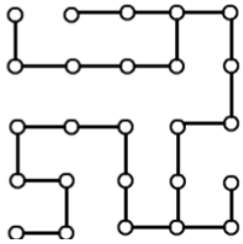
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
#grafi famosi di piccole dimensioni

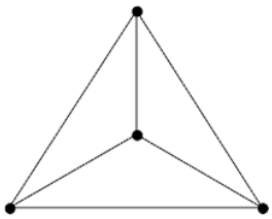
petersen= nx.petersen_graph()
print("Petersen nodi:\t",petersen.nodes())
print("Petersen archi:\t",petersen.edges())
print()
tutte= nx.tutte_graph()
print("Tutte nodi:\t", tutte.nodes())
print("Tutte archi:\t", tutte.edges())
print()
```

```
Petersen nodi:   [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
Petersen archi:  [(0, 1), (0, 4), (0, 5), (1, 2), (1, 6), (2, 3), (2, 7), (3, 4), (3, 8), (4, 9), (5, 7), (5, 8), (6, 8), (6,
9), (7, 9)]

Tutte nodi:      [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29,
30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45]
Tutte archi:     [(0, 1), (0, 2), (0, 3), (1, 4), (1, 26), (2, 10), (2, 11), (3, 18), (3, 19), (4, 5), (4, 33), (5, 6), (5, 2
9), (6, 7), (6, 27), (7, 8), (7, 14), (8, 9), (8, 38), (9, 10), (9, 37), (10, 39), (11, 12), (11, 39), (12, 13), (12, 35), (13,
14), (13, 15), (14, 34), (15, 16), (15, 22), (16, 17), (16, 44), (17, 18), (17, 43), (18, 45), (19, 20), (19, 45), (20, 21), (2
0, 41), (21, 22), (21, 23), (22, 40), (23, 24), (23, 27), (24, 25), (24, 32), (25, 26), (25, 31), (26, 33), (27, 28), (28, 29),
(28, 32), (29, 30), (30, 31), (30, 33), (31, 32), (34, 35), (34, 38), (35, 36), (36, 37), (36, 39), (37, 38), (40, 41), (40, 4
4), (41, 42), (42, 43), (42, 45), (43, 44)]
```



Maze

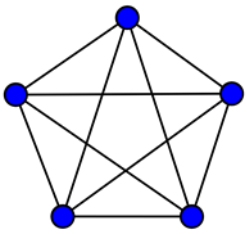


Tet

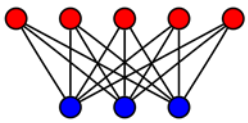
```
#grafi famosi di piccole dimensioni
maze = nx.sedgewick_maze_graph()
print("Maze nodi:\t",maze.nodes())
print("Maze archi:\t",maze.edges())
print()
tet= nx.tetrahedral_graph()
print("Tet nodi:\t", tet.nodes())
print("Tet archi:\t", tet.edges())
```

```
Maze nodi:       [0, 1, 2, 3, 4, 5, 6, 7]
Maze archi:      [(0, 2), (0, 7), (0, 5), (1, 7), (2, 6), (3, 4), (3, 5), (4, 5), (4, 7), (4, 6)]

Tet nodi:        [0, 1, 2, 3]
Tet archi:       [(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
```



Grafo completo



Grafo completo bipartito

PEGASO Università Telematica

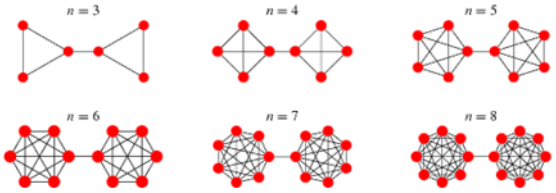
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
#grafi classici

K_5 = nx.complete_graph(5)
print("K_5 nodi:\t",K_5.nodes())
print("K_5 archi:\t",K_5.edges())
print()
K_3_5 = nx.complete_bipartite_graph(3, 5)
print("K_3_5 nodi:\t",K_3_5.nodes())
print("K_3_5 archi:\t",K_3_5.edges())
print()
```

```
K_5 nodi:       [0, 1, 2, 3, 4]
K_5 archi:      [(0, 1), (0, 2), (0, 3), (0, 4), (1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)]

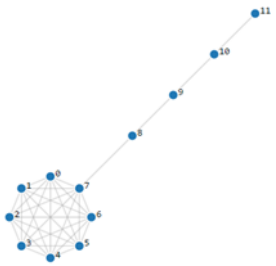
K_3_5 nodi:     [0, 1, 2, 3, 4, 5, 6, 7]
K_3_5 archi:    [(0, 3), (0, 4), (0, 5), (0, 6), (0, 7), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (2, 3), (2, 4), (2, 5), (2,
6), (2, 7)]
```



Barbell

```
barbell = nx.barbell_graph(5, 0)
print("barbell nodi:\t",barbell.nodes())
print("barbell archi:\t",barbell.edges())
```

```
barbell nodi:   [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
barbell archi:  [(0, 1), (0, 2), (0, 3), (0, 4), (1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4), (4, 5), (5, 6), (5, 7), (5,
8), (5, 9), (6, 7), (6, 8), (6, 9), (7, 8), (7, 9), (8, 9)]
```



Lollipop

```
#grafi classici
lollipop = nx.lollipop_graph(10, 20)
print("lollipop nodi:\t",lollipop.nodes())
print("lollipop archi:\t",lollipop.edges())
print()
```

```
lollipop nodi:  [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29]
lollipop archi: [(0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (0, 7), (0, 8), (0, 9), (1, 2), (1, 3), (1, 4), (1, 5), (1,
6), (1, 7), (1, 8), (1, 9), (2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (2, 9), (3, 4), (3, 5), (3, 6), (3, 7), (3, 8), (3,
9), (4, 5), (4, 6), (4, 7), (4, 8), (4, 9), (5, 6), (5, 7), (5, 8), (5, 9), (6, 7), (6, 8), (6, 9), (7, 8), (7, 9), (8, 9), (9,
10), (10, 11), (11, 12), (12, 13), (13, 14), (14, 15), (15, 16), (16, 17), (17, 18), (18, 19), (19, 20), (20, 21), (21, 22), (2
2, 23), (23, 24), (24, 25), (25, 26), (26, 27), (27, 28), (28, 29)]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Alcuni **algoritmi** in Networkx funzionano solo per *grafi non orientati*, mentre altri non sono definiti per i *grafi orientati*. Per trattare un grafo orientato come uno non orientato per alcune misure è possibile utilizzare il metodo ***Graph.to_undirected***() (per l'inverso esiste il metodo *to_directed*()).

Gli operatori principali dei grafi sono i seguenti:

- **subgraph**(nodi) – questo metodo del grafo restituisce il sottografo (inclusi archi) dei nodi indicati come argomento;
- **union**(G1, G2) – restituisce unione dei grafi G1 e G2 (che devono essere disgiunti altrimenti viene sollevata un'eccezione);
- **compose**(G1, G2) – restituisce l'unione dell'insieme dei nodi e dell'insieme degli archi. A differenza di *union*() i due grafi non devono essere disgiunti, anche se è raccomandato che i grafi siano o entrambi orientate o entrambi non orientati;
- **cartesian_product**(G1, G2) – restituisce il prodotto cartesiano dei grafi G1 e G2;
- **create_empty_copy**(G) – restituisce una copia del grafo G con tutti gli archi rimossi;
- **to_undirected**(G) – restituisce una rappresentazione non orientata del grafo G;
- **to_directed**(G) – restituisce una rappresentazione orientata del grafo G.

Per comprendere al meglio come utilizzare gli operatori appena citati, si propone un esempio di uso del metodo **subgraph**().

Attraverso il codice seguente, si genera il grafo completo di 5 nodi (nodi 0,1,2,3,4 tutti collegati tra loro). Si richiede il sottografo composto dai soli nodi 0,1,2. I nodi del sottografo saranno, ovviamente, solo 0,1,2, mentre gli archi saranno (0, 1), (0, 2), (1, 2).

```
G = nx.complete_graph(5)
print(G.edges())
H = G.subgraph([0, 1, 2])
list(H.edges())
```

```
[(0, 1), (0, 2), (0, 3), (0, 4), (1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)]
```

```
[(0, 1), (0, 2), (1, 2)]
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Zinoviev D. (2017). Data science con Python. Apogeo.
- Department of Computer Science and Technology, University of Cambridge - https://www.cst.cam.ac.uk/
- Networkx - https://networkx.org/

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).